

Sistemas Distribuidos
(75.29) Curso Roca

TP Wiki Analytics
Diseño de Arquitectura Distribuida

13 de octubre de 2026

Celeste Zeolla
112193

Raquel Dávila
112002

Tom Pinargote
111689

1. Consideraciones

1.1. Problema y alcance

El sistema propuesto analiza conjuntos de artículos de Wikipedia en inglés y francés para obtener información sobre sus fechas, secciones, imágenes y vocabulario. La entrada proviene de un dataset en formato Parquet disponible en Kaggle. Cada artículo contiene datos estructurados y un *abstract*, es decir, el texto introductorio que resume su contenido.

El problema consiste en procesar un corpus potencialmente mayor que la memoria disponible y atender a varios clientes concurrentemente. El diseño distribuye el trabajo entre computadoras y procesa los datos incrementalmente, preservando la exactitud de las agregaciones y el aislamiento de cada ejecución. Agregar recursos debe permitir aumentar tanto la cantidad de clientes atendidos como el volumen de entrada por cliente.

El cliente dispone de los artículos y los envía incrementalmente al sistema; la ingesta, la selección de campos y el cálculo de las consultas quedan a cargo del servicio. Las cinco consultas comparten la entrada de una ejecución. El cliente también recibe resultados durante el envío y comunica explícitamente el fin de la entrada. El alcance admite una única ejecución por cliente; cada ejecución tiene una identidad que permite separar sus datos, estado y resultados de los de otros clientes.

1.2. Resultados requeridos

Para los artículos recibidos en cada ejecución se deben resolver las siguientes consultas. Sus actores y escenarios se desarrollan en la sección 2.

Q1. URL y fecha de modificación de los artículos en inglés cuya fecha de última modificación corresponde a 2020 o a un año anterior.

Q2. Nombre, URL del artículo y bytes de su imagen para los artículos hasta 2020 inclusive que contienen una sección denominada `biography` o `biographie`. La búsqueda comprende ambos idiomas y compara el nombre completo de la sección en minúsculas. El sistema obtiene el contenido de las imágenes referenciadas mediante Wikimedia.

Q3. Palabras únicas compartidas entre los abstracts de ambos idiomas: cada palabra debe aparecer en al menos un artículo en inglés y en al menos uno en francés.

Q4. Hasta 20 pares de palabra y cantidad, correspondientes a las palabras con mayor conteo entre todos los abstracts de la ejecución.

Q5. URL y cantidad de palabras del abstract de los artículos desde 2021 cuyo conteo supera estrictamente el promedio de los abstracts de los artículos hasta 2020 inclusive.

Los cortes temporales utilizan la fecha de última modificación (`date_modified`). Para el análisis de vocabulario se normalizan las palabras a minúsculas y se excluyen tokens numéricos y *stopwords*, palabras triviales de las listas de inglés y francés utilizadas por el sistema. En Q4 y Q5 cada palabra distinta se cuenta una sola vez por abstract; esta regla no elimina registros repetidos de la entrada. Por ejemplo, dos apariciones de una misma palabra válida en un abstract aportan una unidad a Q4 y al conteo de ese abstract en Q5.

Q1, Q2 y Q3 permiten entregar resultados parciales. Q4 y Q5 requieren haber procesado toda la entrada del cliente: el top global y el promedio histórico dependen del conjunto completo. Las respuestas pueden transmitirse en lotes acotados, acompañados de una finalización explícita de cada consulta, incluso cuando el resultado sea vacío.

1.3. Requisitos que condicionan el diseño

- **Concurrencia y aislamiento.** Cada ejecución conserva entradas, estado y resultados aislados al atender varios clientes concurrentemente. No se exige paralelismo.

- **Determinismo.** Entradas iguales, bajo el mismo contrato de procesamiento, deben producir los mismos resultados aunque cambien el orden de llegada o la intercalación entre clientes. En Q4 se ordena por cantidad descendente y se desempata por palabra normalizada en orden lexicográfico ascendente.
- **Escalabilidad entre computadoras.** El trabajo debe poder distribuirse entre más elementos de cómputo. No se exige elasticidad durante una ejecución; la propuesta fija las particiones y sus propietarios antes de iniciarla, como se explica en la sección 5.1.
- **Recursos acotados.** El procesamiento debe limitar memoria, lotes y operaciones pendientes. Se permiten archivos temporales locales para conservar estado que exceda la RAM y se prohíbe utilizar una base de datos.
- **Tecnología y comunicación.** La implementación se realizará en Go y utilizará un middleware sobre RabbitMQ que abstraiga la comunicación basada en grupos mediante colas y exchanges. La propuesta utiliza TCP entre el cliente y Gateway, AMQP para los mensajes internos y HTTPS para obtener metadatos y bytes de imágenes de Wikimedia.

1.4. Finalización y fallos

El fin de entrada del cliente indica que no enviará más artículos. Cada etapa debe completar su trabajo pendiente antes de propagar la finalización a sus destinos. Recibir el EOF de un productor no implica haber recibido el de todos los productores esperados ni haber procesado todos sus datos. Esta distinción permite que las agregaciones se cierren con resultados completos.

Se requiere un apagado ordenado ante SIGTERM: coordinar el trabajo pendiente, las confirmaciones y el cierre de archivos y conexiones, o informar una cancelación explícita si no se puede completar la ejecución. Ante una caída abrupta, el alcance confirmado es abortar las ejecuciones afectadas. Un nuevo intento utiliza otra identidad de ejecución y retransmite toda la entrada; reiniciar un contenedor no recupera por sí solo el trabajo anterior. La propuesta no ofrece recuperación transparente ni alta disponibilidad.

1.5. Validación y alcance del documento

La corrección se verificará mediante un notebook serial de referencia, adaptado a las mismas listas de stopwords y reglas de conteo del sistema, incluida la deduplicación por abstract de Q5 y el desempate de Q4. Sobre un subconjunto fijo del dataset se compararán los resultados esperados con ejecuciones de un solo cliente y de varios clientes concurrentes. Variar el orden de entrada permitirá comprobar que el resultado no depende del orden de procesamiento; las pruebas de Q2 deben utilizar imágenes de referencia reproducibles para comparar también sus bytes.

Este informe presenta una propuesta de arquitectura mediante vistas $4 + 1$ y diagramas de responsabilidades, interacción y despliegue.

2. Vista de casos de uso

Esta vista presenta los objetivos del cliente y las interacciones externas del sistema. Aporta los casos y escenarios de la vista +1 del modelo 4+1. Los cinco casos corresponden a las consultas Q1–Q5 del DAG y comparten los artículos enviados en una ejecución. Su representación por separado identifica los resultados esperados; no prescribe cinco conexiones ni cinco envíos del dataset.

2.1. Actores y alcance

El **Cliente** es la aplicación que dispone del dataset, envía los artículos en inglés y francés y recibe los resultados y el estado de la ejecución. Es el actor principal de los cinco casos. La dirección de su Gateway está configurada previamente, como se describe en la sección 6.2.

Wikimedia es el actor externo de apoyo de CU2: proporciona los metadatos y el contenido de las imágenes referenciadas por los artículos. El límite del sistema incluye Gateway, ResultCollector, los workers y RabbitMQ; estos componentes internos se desarrollan en las otras vistas. Las asociaciones de la figura 1 indican participación, sin representar un orden de ejecución ni canales de comunicación internos.

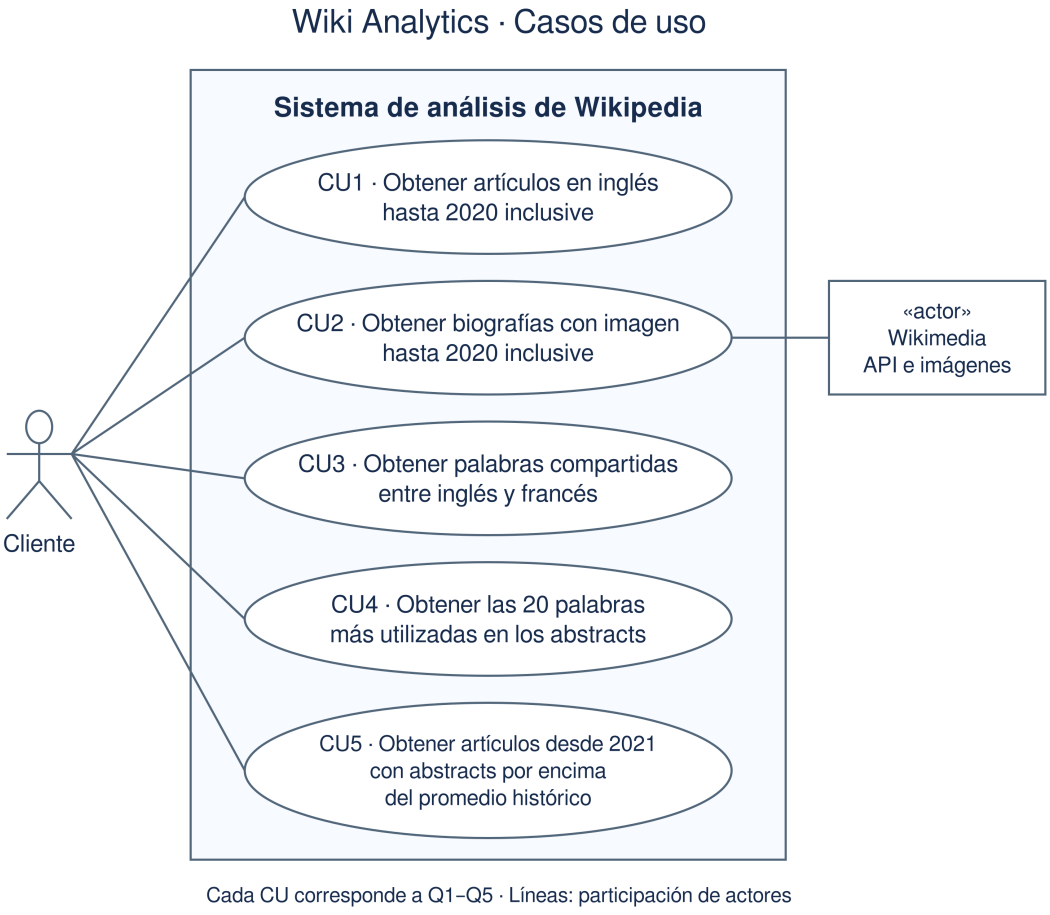


Figura 1: Casos de uso del sistema de análisis de Wikipedia. El Cliente obtiene los resultados de Q1–Q5; Wikimedia participa en la obtención de imágenes para Q2.

2.2. Resultados de los casos de uso

Los cortes temporales se aplican a la fecha de última modificación (`date_modified`). Cada caso opera sobre los datos de su ejecución, sin combinar los artículos de clientes diferentes.

CU1 (Q1). Obtener artículos en inglés hasta 2020 inclusive. El cliente obtiene la URL y la fecha de modificación de los artículos en inglés que cumplen el corte temporal.

CU2 (Q2). Obtener biografías con imagen hasta 2020 inclusive. El cliente obtiene nombre, URL del artículo y bytes de imagen para los artículos que cumplen el corte temporal y contienen una sección denominada `biography` o `biographie`, comparada en minúsculas. El sistema obtiene las imágenes de Wikimedia cuando existe una referencia. La salida acordada sigue el enunciado: incluye la URL del artículo en lugar de la fecha propuesta en V1.

CU3 (Q3). Obtener palabras compartidas entre inglés y francés. El cliente obtiene las palabras únicas presentes en al menos un abstract en inglés y al menos uno en francés. Se utiliza el vocabulario normalizado de los abstracts, excluyendo stopwords y tokens numéricos.

CU4 (Q4). Obtener las 20 palabras más utilizadas en los abstracts. El cliente obtiene hasta 20 pares (palabra, cantidad), correspondientes a las palabras con mayor conteo entre todos los abstracts de la ejecución. Se excluyen stopwords y cada palabra aporta una sola unidad por abstract.

CU5 (Q5). Obtener artículos desde 2021 con abstracts por encima del promedio histórico. El cliente obtiene la URL y el conteo de palabras de los artículos desde 2021 cuyo abstract supera estrictamente el promedio de los abstracts de artículos hasta 2020 inclusive. Al igual que en Q4, se excluyen stopwords y se cuenta cada palabra distinta una sola vez por abstract.

2.3. Escenario común de ejecución

La precondición es que el cliente disponga del dataset y pueda establecer la conexión con el Gateway configurado. El escenario satisfactorio comprende:

1. El cliente inicia una ejecución y envía incrementalmente sus artículos al sistema.
2. El sistema procesa las cinco consultas con estado aislado para esa ejecución. El cliente recibe resultados mientras continúa enviando datos; Q1, Q2 y Q3 permiten entregas parciales.
3. El cliente indica el fin de la entrada. El sistema completa el procesamiento pendiente; Q4 y Q5 entregan resultados después de procesar toda la entrada, aunque su salida pueda transmitirse en lotes.
4. El cliente recibe la finalización de cada consulta y de la ejecución. Una finalización exitosa indica que se entregaron todos los resultados correspondientes, incluyendo consultas sin coincidencias.

Variantes y terminaciones no exitosas. En CU2, una referencia de imagen ausente se informa explícitamente. Si existe una referencia pero su descarga falla definitivamente, Q2 se informa como fallida y los resultados parciales no se presentan como una respuesta completa. En CU5, si no hay abstracts históricos computables, se devuelve un resultado vacío con el estado *sin base histórica*.

Una caída abrupta aborta las ejecuciones afectadas. Un nuevo intento utiliza otra identidad de ejecución y retransmite toda la entrada; el cliente no acumula los resultados del intento fallido. Ante SIGTERM, el sistema coordina la finalización de las ejecuciones activas o notifica una cancelación explícita, sin anunciar éxito para una ejecución incompleta. Las condiciones operativas se detallan en la vista física y la gestión de estado en la sección 5.2.

3. Vista Lógica

Con el fin de visualizar qué compoenentes existirán en el sistema y de qué se encargarán, realizamos un *Dyrected Acyclic Graph* de ahí sus sigas *DAG*:

3.1. DAG

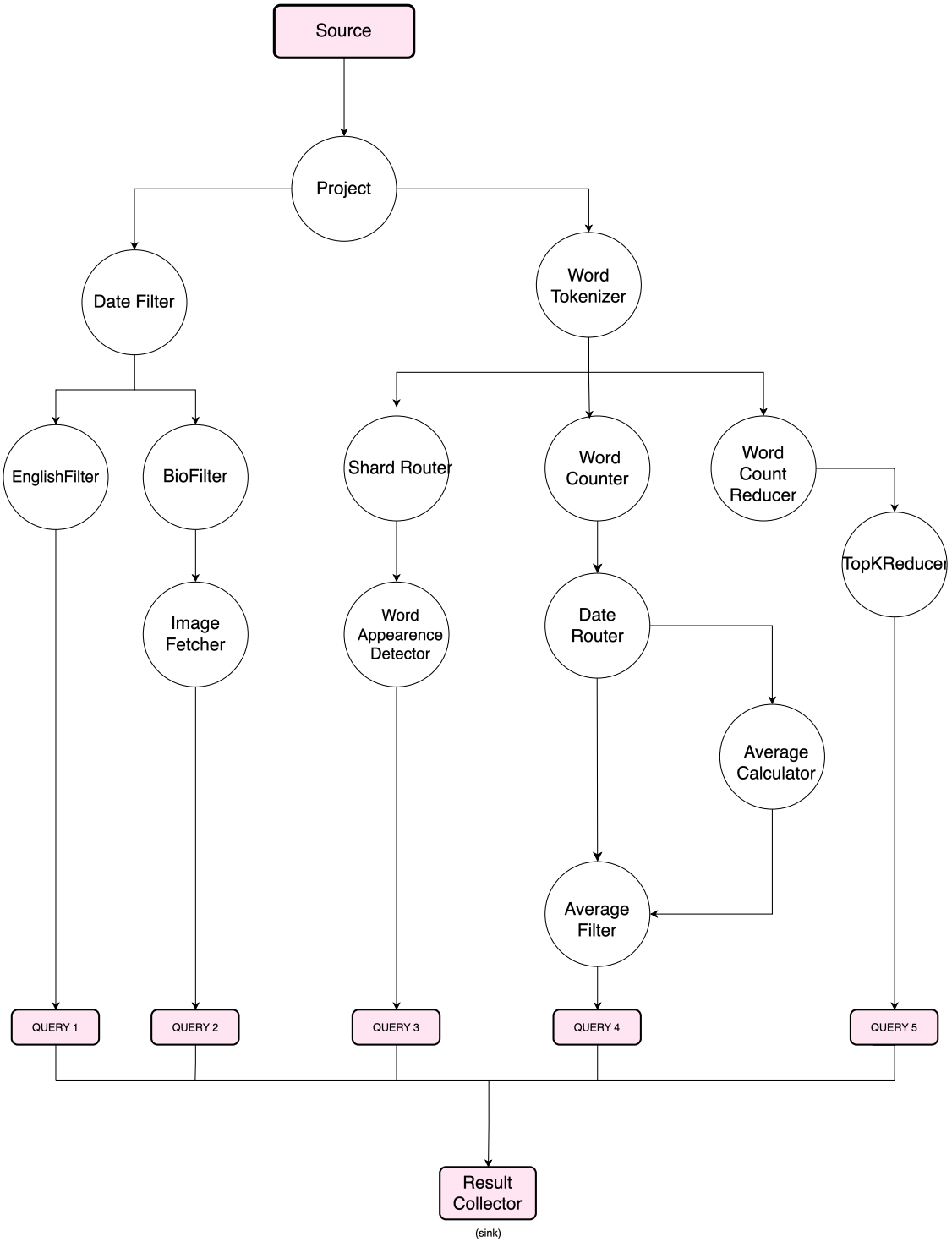


Figura 2: DAG del flujo de resolución de cada consulta del sistema

A continuación detallamos algunas funcionalidades:

- **Project**: Recibe las filas del dataset tal cual el cliente las envía, y se encarga de filtrar las columnas que no serán necesarias para resolver alguna de las cinco consultas.
- **Date Filter y Word Tokenizer**: Dado a que hay queries que comparten este preprocesamiento, aprovechamos en usar los mismos nodos y no hacer el mismo procesamiento para cada flujo.
- **Result Collector**: Si bien, las primeras 3 consultas envían resultados parciales es necesario llevar cuenta de si la petición inicial del cliente terminó, esto es cuando las cinco consultas finalizan, este nodo se encargará de notificar el fin de los clientes.

4. Vista de desarrollo

4.1. Diagrama de paquetes

A la hora de metenernos con la implementación, dividiremos el código en los siguientes módulos:

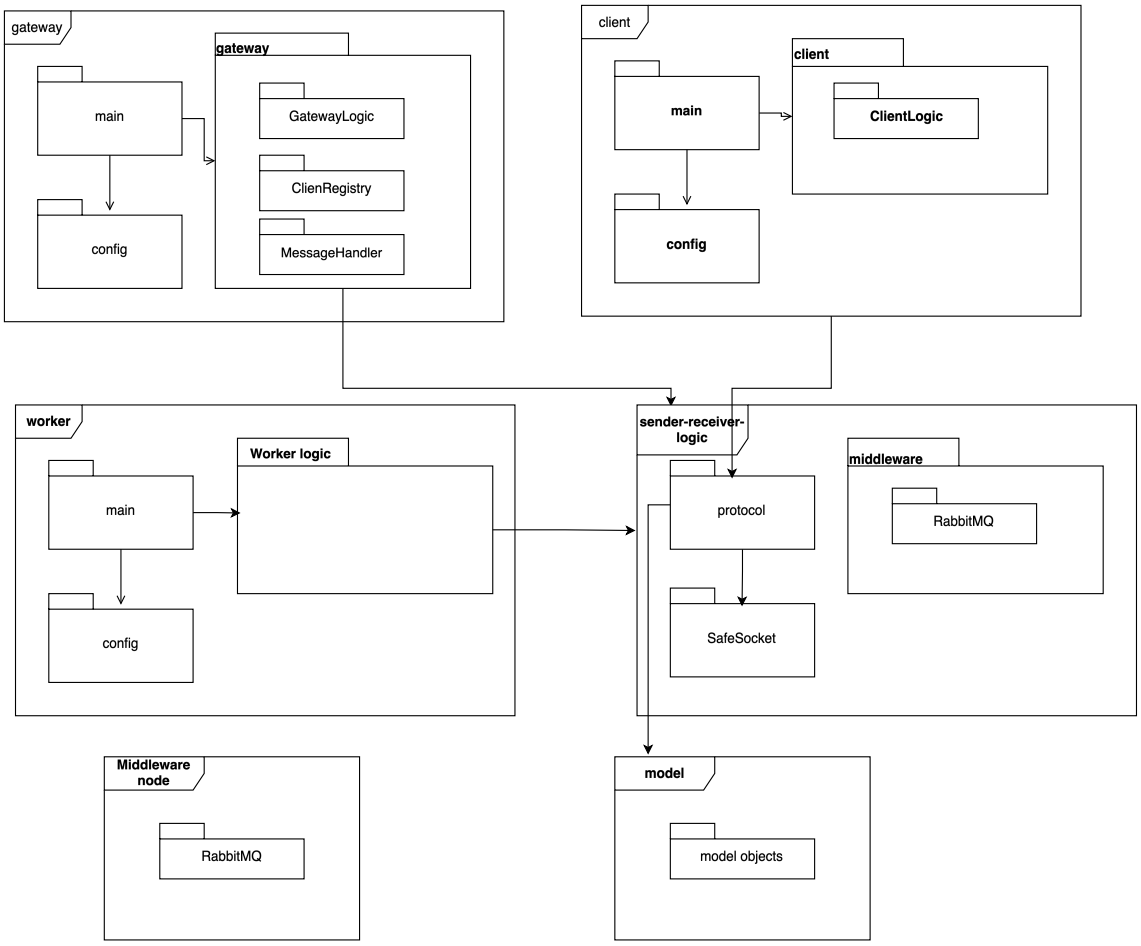


Figura 3: Organización del proyecto

Donde, en particular cada uno representa:

- **Gateway**: Es el servidor central del sistema, y es el único punto de entrada y salida para la información de todos los clientes y repsuestas a emitir.
- **Client**: Posee la lógica de conexión de los clientes y de extracción de la información del datset, como así también el *batching* de la misma.
- **Worker**: Es una abstracción de cada uno de los workers representados como nodos en el *DAG* (ver 3.1).
- **Model**: Contiene los objetos del dominio del problema, que se usarán en el protocolo y por ende en los workers para hacer el manejo de datos del dataset.
- **Sender-Receiver-Logic**: En este módulo se tienen los componentes necesarios para la comunicación entre nodos, esto es la implementación del middleware,el protocolo de mensajes entre workers, cliente-servidor y por último la implementación del socket seguro.

- **Sender-Receiver-Logic**: En este módulo se tienen los componentes necesarios para la comunicación entre nodos, esto es la implementación del middleware,el protocolo de mensajes entre workers, cliente-servidor y por último la implementación del socket seguro.
- **Middleware Node**: Este nodo permite el uso del middleware de RabbitMQ, para usar dicho framework.

5. Vista de procesos

5.1. Particionado y aislamiento de ejecuciones

Una partición, o *shard*, es una porción del trabajo y de su estado. Antes de iniciar cada ejecución se fija la cantidad de particiones por etapa y se configura qué instancia de worker, alojada en qué contenedor, es propietaria de cada una. Para las operaciones por palabra se aplica una función hash estable:

$$p(w) = H(w) \bmod P,$$

donde w es la palabra y P la cantidad de particiones de la etapa. La configuración relaciona el identificador p con el worker y su contenedor; el hash no selecciona por sí solo una máquina. Por ejemplo, con cuatro particiones, un worker puede atender las particiones 0 y 1, y otro las 2 y 3. Los aportes de una misma palabra llegan a su propietario. El trabajo por artículo se reparte por identidad del registro o por lotes de entrada.

Cada partición tiene un único propietario activo de sus acumuladores, archivos y progreso. Un worker puede atender varias particiones; agrupar workers en un contenedor no fusiona sus estados ni sus responsabilidades. Las cantidades y asignaciones permanecen estables durante la ejecución; se redistribuyen antes de nuevas ejecuciones. El manifiesto registra los participantes y destinos esperados para verificar los cierres.

La identidad de ejecución forma parte de las claves de estado, los mensajes y las rutas de archivos. Compartir un contenedor o una partición entre clientes no mezcla sus datos. `TopKReducer`, `Average Calculator` y `ResultCollector` tienen un propietario por ejecución, distribuido entre las instancias disponibles; no existe un único propietario global para todos los clientes. Q4 y Q5 sólo se entregan después de procesar todo el input; Q1–Q3 pueden entregarse parcialmente con seguimiento de cierre.

5.2. Gestión del estado y límites de recursos

Los archivos temporales pertenecen a los workers que necesitan conservar estado y se ubican en montajes locales de sus hosts, separados por ejecución y partición. Su uso depende del rol:

- Q3: `WordAppearenceDetector` conserva presencia por idioma y seguimiento de palabras emitidas, con archivos cuando el vocabulario excede el presupuesto de memoria.
- Q4: `WordCountReducer` vuelca conteos parciales a disco y los combina para mantener memoria acotada ante vocabularios grandes.
- Q5: `Average Filter` guarda en disco los candidatos posteriores a 2020, al menos su URL y conteo de palabras, hasta recibir el promedio histórico definitivo. Luego recorre esos archivos y emite los que superan el umbral. Asume esta responsabilidad sin un nodo aparte. Lo mismo ocurre en `Average Calculator` mientras se esperan todos las longitudes requeridas para calcular el promedio.
- Q2: `ImageFetcher` guarda temporalmente las imágenes descargadas y las lee en fragmentos acotados para enviarlas por RabbitMQ a `ResultCollector`, luego a `Gateway` y finalmente al cliente. Estos archivos de trabajo no constituyen un snapshot persistente ni un servicio de archivos compartidos.

No todos los workers acceden a estos archivos: cada propietario administra los suyos. No se introduce una base de datos ni un almacenamiento compartido central del corpus. Q4 recibe su orden del `TopKReducer`; no se incorpora una ordenación global de las otras salidas en `ResultCollector`.

Cada instancia limita memoria, lotes, entregas pendientes, descargas y disco por ejecución. Frente a clientes lentos, Gateway y ResultCollector mantienen buffers acotados. Las colas y publicaciones pendientes de RabbitMQ también deben tener límites: no se sustituye el disco del collector por acumulación ilimitada en el broker. El seguimiento de entrega y deduplicación debe

liberar estado confirmado sin crecer con toda la salida histórica. Al agotarse una cuota se suspende la admisión o se informa cancelación o fallo. Los archivos temporales se eliminan al completar la entrega o terminar el intento.

5.3. Diagramas de actividades

En esta sección se presentan algunos diagramas que ilustran cómo actuarán los nodos en conjunto.

5.3.1. Consulta n°1

Para la primer funcionalidad, mostramos como el nodo de *Date Filter* elimina del pipeline aquellos datos coyo año sea mayor al 2020, y propaga el EOF sin una lógica adicional. Finalmente, el último filtro se queda únicamente con la data en inglés

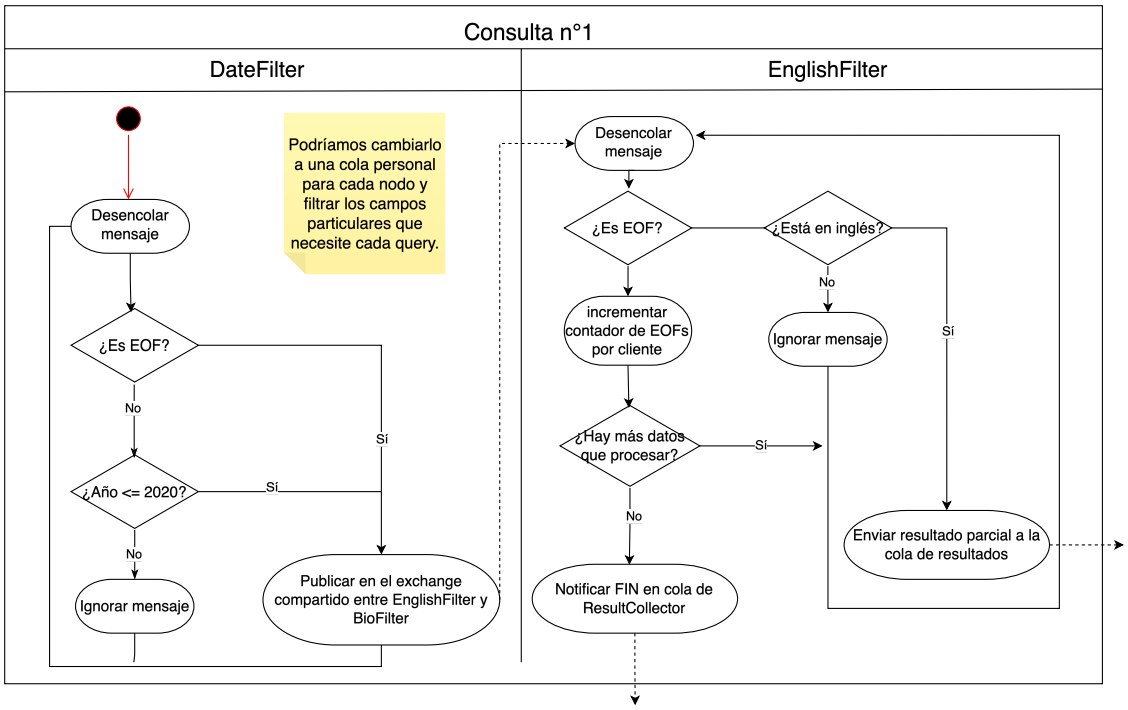


Figura 4: Url y fecha de modificación para artículos escritos en inglés hasta 2020 inclusive.

5.3.2. Consulta n°2

Siguiendo el flujo del DAG, no volvemos a repetir el nodo de *Date Filter* para el diagrama de la siguiente consulta que sí forma parte del pipeline, pero para simplificar no lo agregamos.

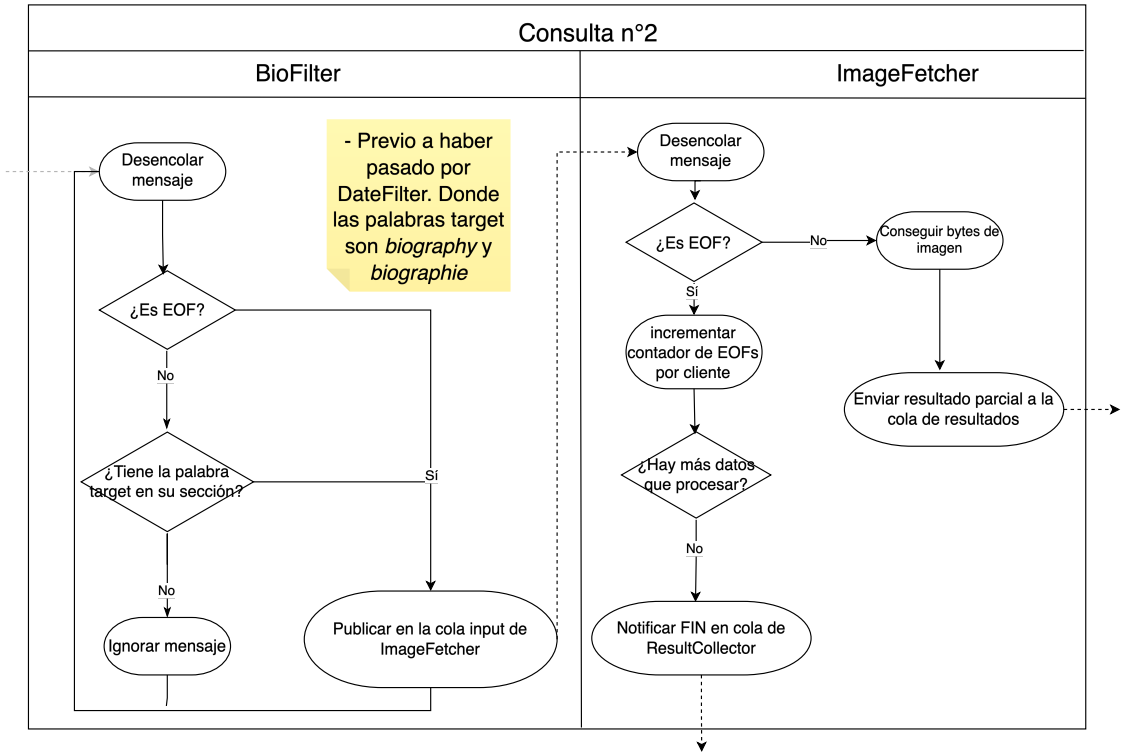


Figura 5: Nombre, url e imágen (bytes) de artículos escritos hasta 2020 inclusive que incluyan "biography" o "biographie" entre sus secciones.

5.3.3. Consulta n°3

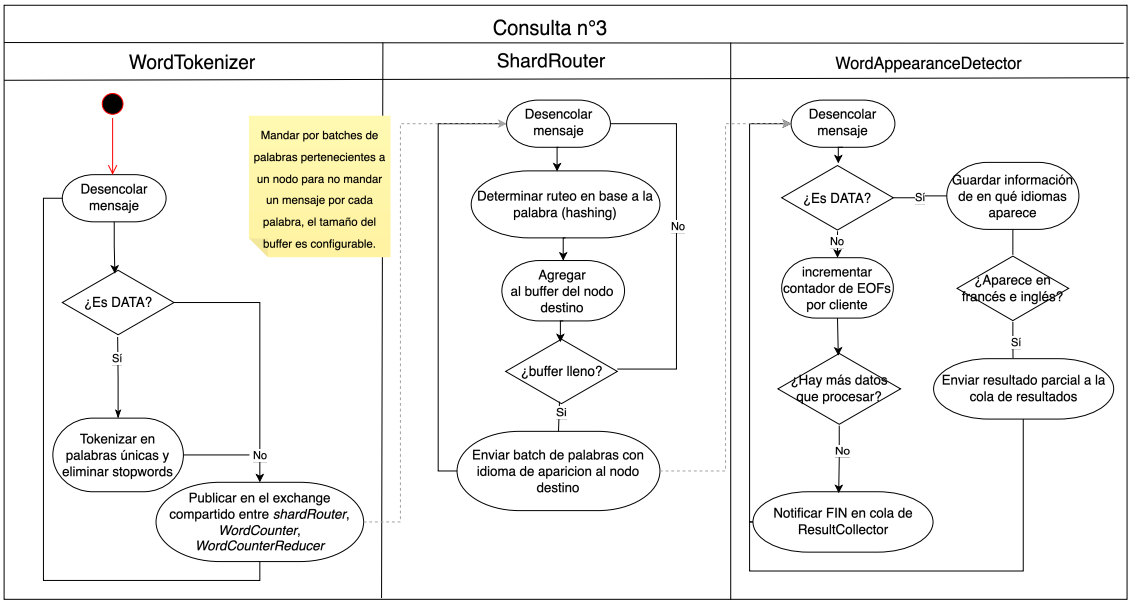


Figura 6: Listado de palabras únicas compartidas entre cualquier artículo escrito en inglés y cualquier artículo escrito en francés.

5.3.4. Consulta n°4

La figura 7 sigue el estilo de carriles de la consulta 3. El flujo comienza en `WordTokenizer`, continúa en `WordCountReducer` y finaliza en `TopKReducer`. Se procesan los abstracts de ambos idiomas, sin filtro por fecha. `WordTokenizer` normaliza el texto, elimina stopwords y obtiene las palabras únicas de cada abstract: cada palabra aporta una unidad, aunque aparezca varias veces en ese abstract. Los registros repetidos en el input conservan sus aportes. Se forman lotes acotados por destino, utilizando el hash estable y las asignaciones descriptas en la sección 5.1.

`WordCountReducer` acumula los aportes de cada palabra por ejecución y partición, con archivos temporales cuando el vocabulario supera el presupuesto de memoria. Un cierre completo exige recibir el EOF de cada productor esperado y haber procesado todos sus datos; no alcanza con recibir un EOF ni encontrar la cola vacía. Los cierres repetidos se registran una sola vez. Antes de propagar su EOF, `WordTokenizer` publica todos los lotes pendientes, incluso los que no alcanzaron el tamaño máximo. Cada destino recibe un cierre aun cuando no haya recibido datos.

Al cerrar su entrada, cada partición combina sus conteos y envía a `TopKReducer` hasta 20 pares de palabra y cantidad, seguidos de su EOF. Tanto el top local como el global usan cantidad descendente y, para empates, palabra normalizada en orden lexicográfico ascendente, según el criterio confirmado por el equipo. Como todos los aportes de una palabra pertenecen a una misma partición, los tops locales contienen los candidatos necesarios para obtener el top global exacto; no se seleccionan tops sobre conteos incompletos. `TopKReducer` mantiene hasta 20 candidatos por ejecución y espera el cierre de todas las particiones y el procesamiento de sus tops antes de enviar el resultado ordenado y el FIN de Q4 a `ResultCollector`. Si existen menos de 20 palabras válidas, devuelve todas; si no hay ninguna, devuelve una lista vacía con su cierre. Q4 no entrega resultados parciales al cliente. Las flechas discontinuas representan mensajes internos mediante RabbitMQ/AMQP; el fin de actividad corresponde a esa ejecución, no al apagado del worker.

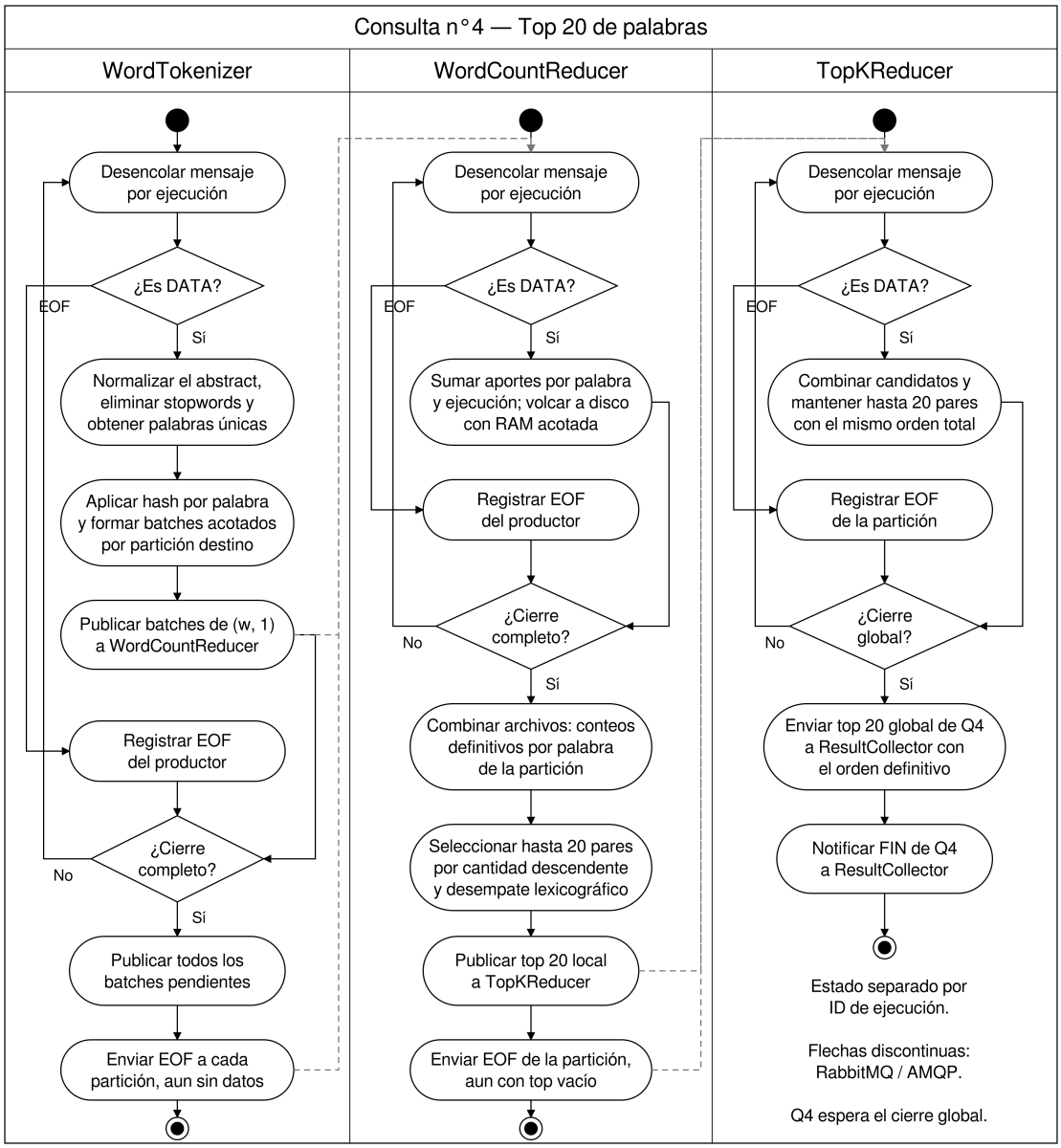


Figura 7: Top 20 de palabras de los abstracts: se excluyen stopwords y cada palabra se cuenta una sola vez por abstract.

5.4. Consulta nº5

Como ya fue explicado en los diagramas de las queries anteriores, para la query 5 no incluimos el `WordTokenizer`, a pesar de que este forma parte del flujo de la consulta según lo visto en el DAG. El flujo observado en el diagrama, entonces, comienza en `WordCounter`, que cuenta las palabras de cada abstract y publica el registro hacia `DateRouter`.

`DateRouter` clasifica cada registro según su fecha. Los artículos hasta 2020 inclusive se publican hacia `AverageCalculator`, porque son los que se necesitarán para calcular el promedio. Los artículos desde 2021 se publican directamente hacia `AverageFilter`, porque son los candidatos a filtrar. Una vez se confirma que no se recibirán más datos, se notifica del EOF tanto al AverageCalculator como al `AverageFilter`.

`AverageCalculator` va guardando los conteos recibidos y una vez se cerciora de que hayan llegado todos, procede a calcular el promedio. Es entonces que publica el promedio obtenido hacia `AverageFilter`, que acumula los conteos recibidos por `DateRouter` hasta recibir este dato. A partir de entonces, este último nodo comienza a filtrar cada documento según si la longitud de su abstract supera el promedio o no, para finalmente publicarlos hacia el `ResultCollector`.

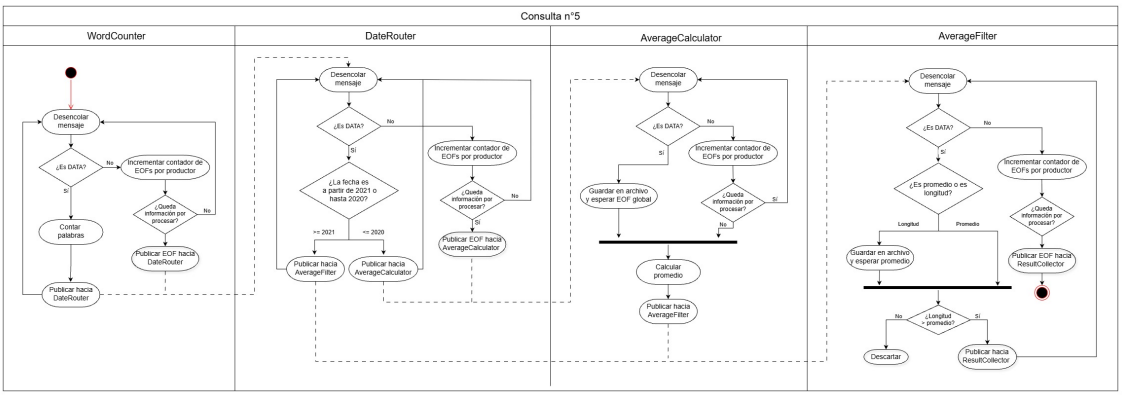


Figura 8: Artículos publicados de 2021 en adelante cuyo abstract tiene una longitud superior a la del promedio de los abstracts de los artículos publicados hasta 2020

5.4.1. Result Collector

Finalmente, mostramos cómo funcionaría el nodo que comunica al usuario los resultados parciales o totales.

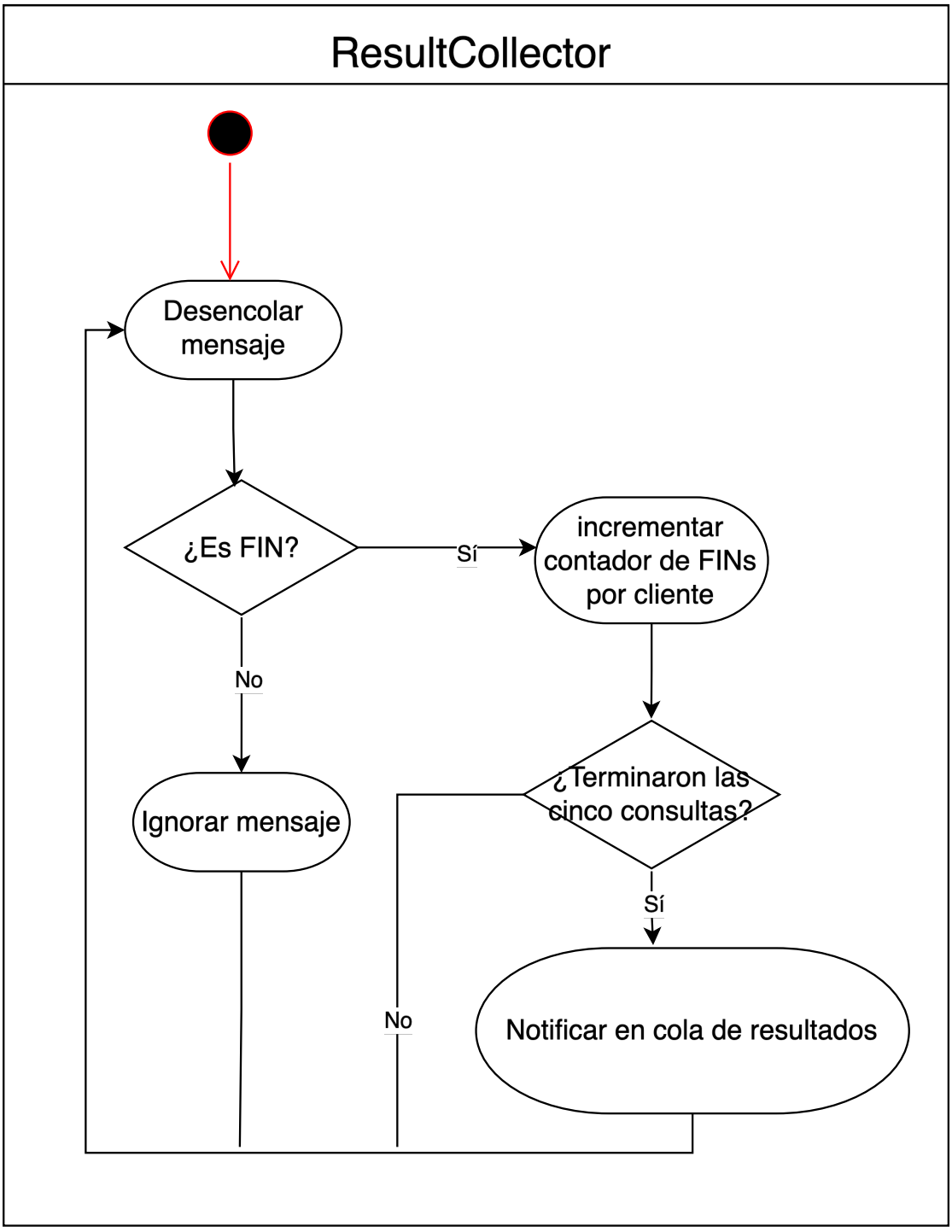


Figura 9: Resolución de primer consulta

5.5. Diagrama de secuencia

Para mostrar una idea general de cómo es que los procesos interactuán hicimos un diagrama de secuencia node se muestra todo el flujo del pipeline y el procesamiento en streaming de los Workers de forma abstracta y de los de más nodos de forma explícita.

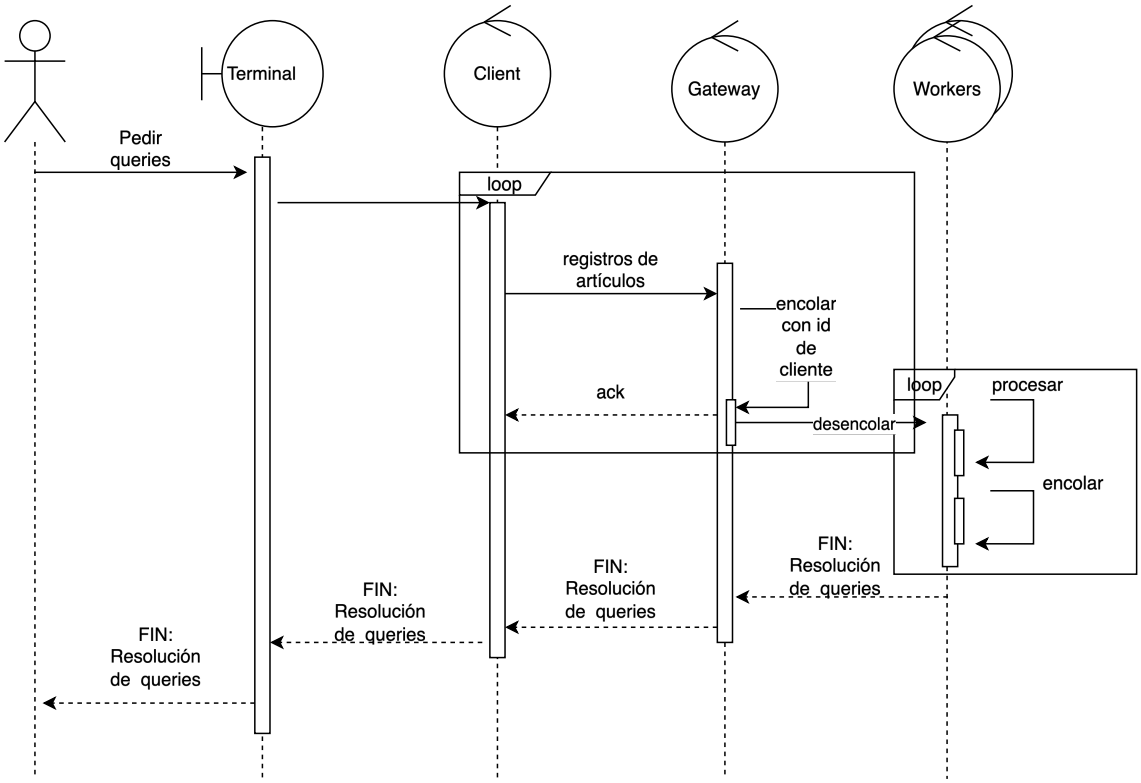


Figura 10: Interacción entre procesos

6. Vista física

La vista física del modelo $4+1$ describe la distribución del sistema entre equipos, los entornos de ejecución y sus enlaces de comunicación. Presentamos el despliegue de la propuesta actual del DAG.

6.1. Diagrama de Robustez

A continuación, se presenta el diseño planteado en el DAG a modo de diagrama de robustez.

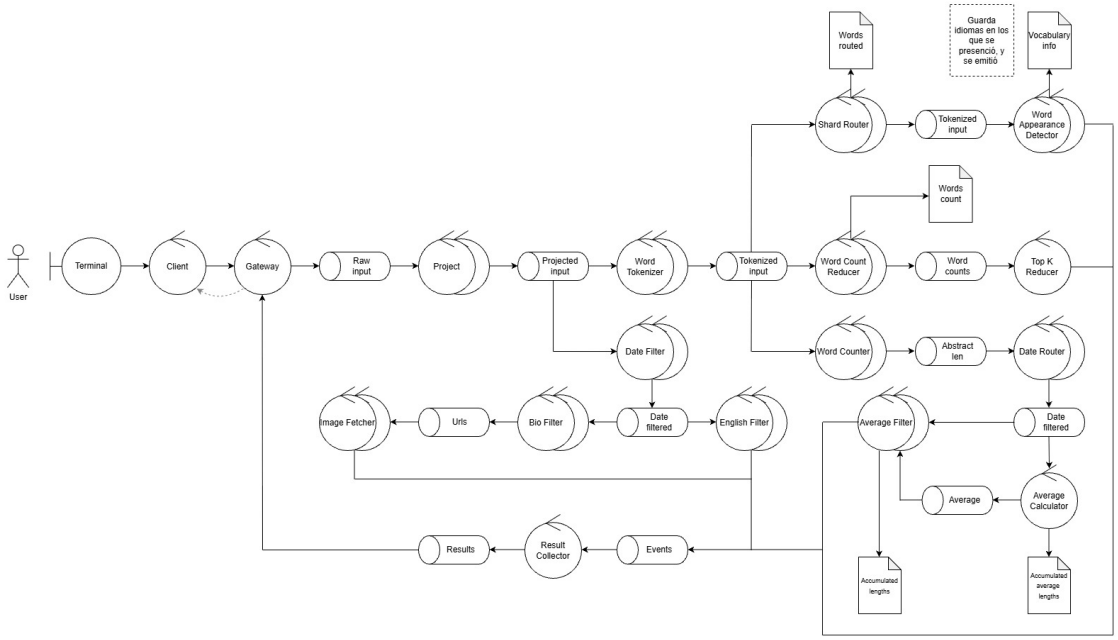


Figura 11: Diagrama de robustez del sistema planteado

El usuario inicia la ejecución desde `Terminal`, que se la entrega a `Client`. `Client` envía los artículos a `Gateway`, que los publica en `Raw input` identificados por cliente. `Project` consume desde ahí, descarta las columnas que ninguna consulta necesita y publica en `Projected input`. En ese momento, el flujo se divide dependiendo de la query: las primeras dos consultas partiendo de `DateFilter`, y las últimas tres partiendo de `WordTokenizer`.

Para las queries 1 y 2, `Date Filter` descarta los artículos posteriores a 2020 y publica el resto en `Date filtered`. `EnglishFilter` consume de ese exchange, se queda con los artículos en inglés y envía sus resultados parciales a `Events`. `BioFilter` también consume de ahí, retiene los artículos que tienen una sección de biografía, y luego se descargan las imágenes de estos mediante `ImageFetcher`, para luego enviar los resultados hacia `Events`.

Para la query 3, `WordTokenizer` publica las palabras únicas de cada abstract en `Tokenized input`. `ShardRouter` las agrupa en lotes por partición hasheandolas, registra el ruteo en el archivo `Words routed` y las entrega a las instancias de `WordAppearenceDetector`. Cada instancia conserva en el archivo `Vocabulary info` los idiomas en que apareció cada palabra y cuáles ya emitió. Cuando una palabra aparece en inglés y en francés, la envía como resultado parcial a `Events`.

Para la query 4, `WordCountReducer` consume `Tokenized input` y acumula las unidades por palabra y partición. Vuelca a disco en el archivo `Words count` el vocabulario para no saturar la memoria. Al finalizar, publica su top 20 local en `Word counts`. `TopKReducer` combina los tops de todas las particiones y envía el top 20 global a `Events`.

Para la query 5, `WordCounter` consume `Tokenized input`, calcula la cantidad de palabras de cada abstract y publica en `Abstract len`. `DateRouter` lee esa cola y separa los registros por fecha. Los de 2020 o anteriores van a `AverageCalculator`, que los acumula en el archivo `Accumulated average lengths` y finalmente publica el promedio en la cola `Average`. Los de 2021 en adelante van directamente a `AverageFilter`, que los guarda en el archivo `Accumulated lengths` hasta recibir el promedio desde `Average`. Con el promedio definitivo, envía a `Events` únicamente los artículos que lo superan.

Todas las consultas terminan pasando sus resultados a `Events`, de donde consume `ResultCollector`, que lleva la cuenta de los FIN por cliente. Cuando terminaron las cinco consultas, notifica el fin de la ejecución. Los resultados se publican en `Results` y de allí llegan finalmente al `Gateway`.

6.2. Despliegue

Cada host del servicio ejecuta un motor Docker y aloja los contenedores correspondientes a sus responsabilidades. La figura 12 representa una distribución ilustrativa: los hosts A y B muestran cómo repartir el trabajo entre computadoras, sin fijar una cantidad mínima ni máxima de equipos. Las cajas de ingesta y workers agrupan procesos asignados a cada host; no representan un único contenedor. Estos procesos pueden repartirse entre varios contenedores o agruparse en uno, según la configuración de despliegue. Un rol lógico del DAG tampoco exige un proceso ni una máquina exclusivos.

Distribución de responsabilidades. Los clientes conservan el dataset de entrada y envían registros incrementalmente a `Gateway`, sin ejecutar las consultas ni conocer la topología de RabbitMQ. Cada cliente tiene configurada la dirección del `Gateway` con el que se comunica; varios clientes pueden utilizar la misma instancia. Mostrar varios gateways ilustra una opción de escalado, sin incorporar un router ni un balanceador adicional.

En el ejemplo, cada host de borde aloja un contenedor `Gateway` y otro `ResultCollector`. El primero administra la sesión TCP y la admisión; el segundo reúne los resultados de las consultas y sus cierres por ejecución. Su ubicación conjunta es ilustrativa: pueden ejecutarse en hosts diferentes, ya que se comunican mediante RabbitMQ. Cada ejecución identifica el Gateway que conserva su conexión TCP y al que deben encaminarse los resultados. El despliegue base no asigna archivos locales a `ResultCollector`: reenvía lotes acotados y mantiene el seguimiento de entrega y finalización.

El host de mensajería aloja un contenedor RabbitMQ con los exchanges y colas del middleware. Los hosts de cómputo alojan contenedores de ingesta y workers de consultas, con archivos temporales en su propio disco. La tabla siguiente relaciona estos grupos con los nombres de la propuesta actual.

Grupo	**Roles del DAG y ubicación**
Ingesta	`Project`: validación y proyección de los campos necesarios, en hosts de cómputo.
Workers Q1/Q2	`Date Filter`, `EnglishFilter`, `BioFilter` e `ImageFetcher`, en hosts de cómputo.
Workers Q3	`WordTokenizer`, `Shard Router` y `WordAppearenceDetector`: presencia de palabras por idioma.
Workers Q4	`WordTokenizer`, `WordCountReducer` y `TopKReducer`: conteos por palabra y top 20 por ejecución.
Workers Q5	`WordTokenizer`, `Word Counter`, `Date Router`, `Average Calculator` y `Average Filter`: conteos por abstract, promedio histórico y evaluación de candidatos.
Borde	`Gateway` y `ResultCollector`: entrada y salida TCP, sesión y entrega de resultados.

Tanto Q4 como Q5 excluyen stopwords y cuentan cada palabra una sola vez por abstract,

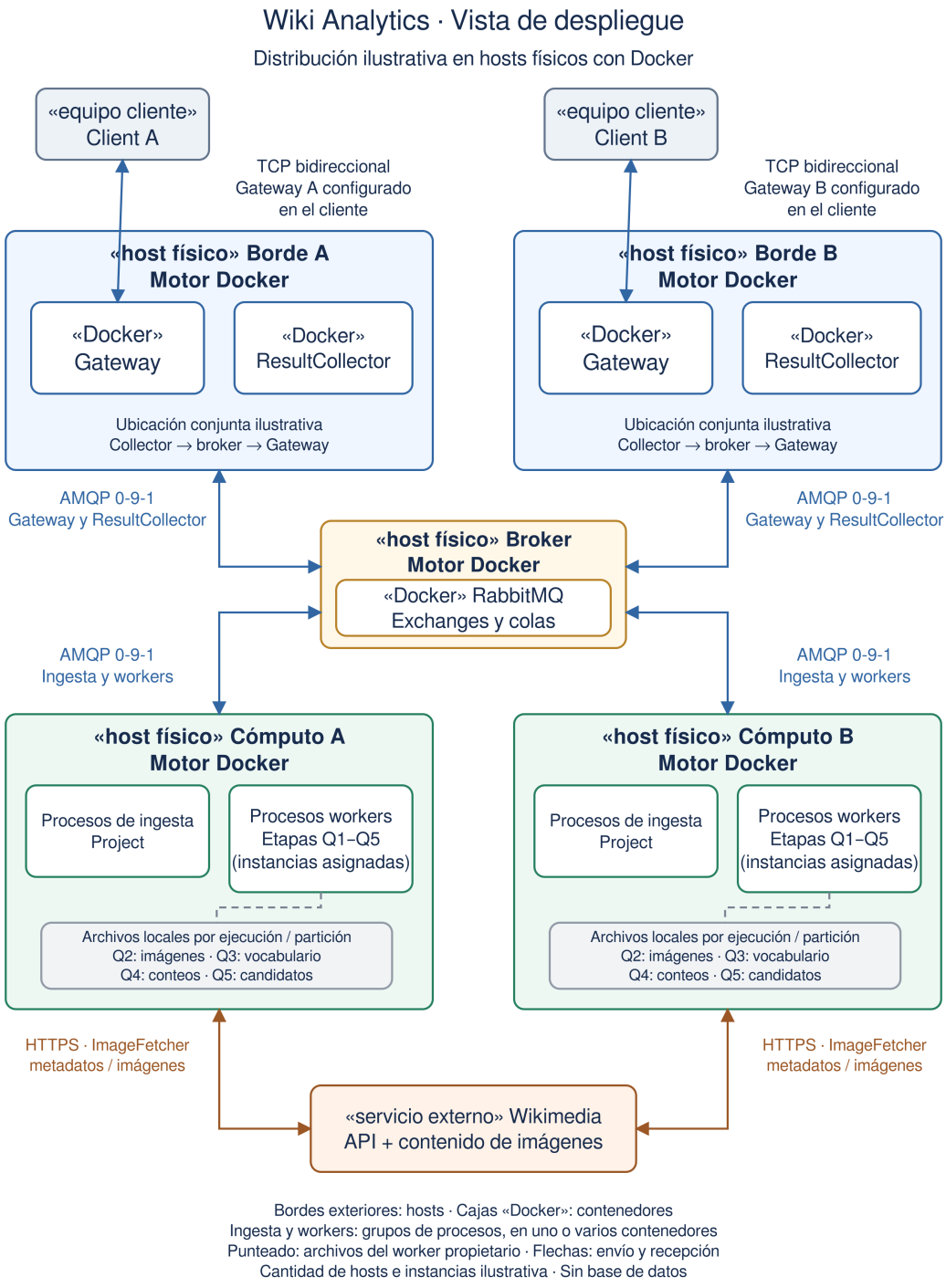


Figura 12: Despliegue ilustrativo del DAG en hosts físicos con contenedores Docker. Las cajas de ingesta y workers agrupan procesos, cuya distribución entre contenedores es configurable. Los enlaces AMQP conectan las instancias con el broker; HTTPS corresponde únicamente a ImageFetcher.

conforme a la decisión actual del equipo (el notebook de referencia no excluye palabras repetidas en Q5).

Enlaces de comunicación. El enlace `Client`–`Gateway` utiliza TCP bidireccional con mensajes delimitados por longitud para transmitir artículos, resultados y fragmentos de imágenes. El cliente debe recibir resultados mientras envía entrada. Los contenedores del servicio intercambian datos y control mediante AMQP 0-9-1 sobre RabbitMQ, incluido el envío de `ResultCollector` a `Gateway`, aunque ambos estén en el mismo host. Los exchanges `topic` enrutan hacia etapas y particiones; la difusión del umbral de Q5 debe alcanzar a cada destinatario mediante colas identificadas.

Sólo los contenedores que alojan `ImageFetcher` necesitan el enlace HTTPS externo para resolver metadatos mediante la API de Wikimedia y obtener los bytes desde la URL de contenido correspondiente. Las imágenes se entregan por el circuito de resultados hasta `Gateway`. Las flechas entre hosts resumen estas conexiones de los contenedores; no agregan comunicación directa entre etapas que evite el middleware.

Asignación de trabajo y almacenamiento local. La configuración de despliegue asigna las particiones a instancias de workers alojadas en contenedores. Esa asignación permanece fija durante cada ejecución; un worker puede atender varias particiones. El criterio de particionado y el aislamiento del estado se describen en la sección 5.1.

Los archivos temporales de Q2–Q5 residen en el disco del host de cómputo, mediante montajes locales accesibles al worker propietario. No se introduce una base de datos ni almacenamiento compartido entre hosts. El despliegue debe proveer memoria y disco acordes con las cuotas de las instancias alojadas; la responsabilidad sobre esos archivos, su ciclo de vida y el control de recursos se detallan en la sección 5.2.

Escalabilidad y operación. Un único Gateway puede atender múltiples clientes concurrentes. Para ampliar la capacidad de entrada y salida se pueden agregar gateways y propietarios de ejecuciones, eventualmente en nuevos hosts de borde. Los clientes se asignan por configuración a esas instancias; no se promete selección transparente ni migración de sesiones entre gateways. Para procesar mayor volumen por cliente se agregan hosts de cómputo y se distribuyen las particiones antes de las nuevas ejecuciones. Los contenedores conservan el mismo DAG y contrato semántico. Persisten límites en el broker, la salida de cada ejecución por su `Gateway` y `ResultCollector`, y las particiones con palabras muy frecuentes. Los recursos se dimensionan según la carga; no se promete crecimiento lineal ilimitado.

Ante SIGTERM se detiene la admisión y se coordina el drenaje de las ejecuciones activas. Los procesos Go deben recibir la señal; el plazo de parada de Docker debe permitir drenar o notificar una cancelación explícita. Se cierran archivos y conexiones después de confirmar las operaciones pendientes, y RabbitMQ se detiene al final del apagado coordinado. No se anuncia éxito al cancelar.

Una caída abrupta de un propietario o la pérdida de sus archivos aborta las ejecuciones afectadas. Una caída de `Gateway` invalida sus sesiones; la del broker invalida las ejecuciones dependientes. El cliente reinicia con una identidad nueva y retransmite todo el input, sin acumular resultados del intento anterior. Reiniciar un contenedor no recupera sus ejecuciones. El broker único por instalación no implica alta disponibilidad.

7. Distribución de trabajo estimada

Componente / Tarea	**Responsable**
query1	Raquel Dávila
query2	Raquel Dávila
query3	Tom Pinargote
query4	Tom Pinargote
query5	Celeste Zeolla
Server	Celeste Zeolla
Middleware	Tom Pinargote
Cliente	Celeste Zeolla
Protocolo y modelo	Raquel Dávila

Cuadro 1: Distribución estimada de tareas por módulo y miembro del equipo.